Phase 4 Plan: Join

A simple explanation of what's next, and how we're going to tackle it — October 2026

What Is Phase 4 About?

Phase 3 proved something important: we can take one FIA penalty decision and find its exact matching moment in real race telemetry. But right now, that result only exists as output printed to a terminal, from a script we ran by hand (`verdict/telemetry/explore_session.py`). If we closed that terminal, the discovery is gone.

Phase 4 is about making that discovery permanent, and repeatable. Instead of re-running a manual investigation every time we want to look at a decision's telemetry, we save the result once, attached to the penalty record it belongs to. From then on, anything that needs it — search, comparisons, a future webpage — can just look it up, instead of re-doing the detective work from Phase 3 every single time.

Why Might This Be Tricky?

Two real wrinkles to plan around, both things we only discovered because we did Phase 3 by hand first:

- **We don't actually store which race a decision came from.** Our parser captures what's inside the document body (Fact, Decision, Reason, etc.), but never the year or Grand Prix name from the header. We only knew “2024 Abu Dhabi” because we remembered it from when the PDF was first shared. Every future decision needs this stored properly, or Phase 4 can't be automated at all.
- **Deciding how much telemetry to actually save.** A full lap of telemetry is hundreds of data points per car. We don't want to save all of it for every decision — just a focused snapshot around the actual incident moment (which Phase 3 already taught us how to find).

The Plan: Steps We'll Take

1. **Close the race-identity gap.** Add a way to capture which race/year a decision belongs to — either by extracting it from the PDF's header, or (simpler, for now) by naming raw PDF files consistently and reading it from the filename. Store it as new fields on the parsed JSON and in the `penalty_decisions` table.
2. **Decide what a “telemetry snapshot” looks like.** Based on what we learned in Phase 3: a short window of Speed, Brake, and position data for both `own_car` and `other_car`, covering a few seconds before and after the matched incident moment — not the whole race.
3. **Turn Phase 3's manual steps into a reusable function.** Everything we did by hand in `explore_session.py` (load the session, get the driver's telemetry, find the sharpest speed drop) becomes one function: give it a parsed decision, get back a telemetry snapshot.

4. **Design where the snapshot gets stored.** The simplest option: a new column on `penalty_decisions` that holds the snapshot as structured data (Postgres can store this directly, no separate table needed yet).
5. **Run it for the one decision we've already proven (Abu Dhabi, Bottas vs. Car 20)**, and confirm what gets saved matches exactly what we found by hand in Phase 3.
6. **Only after that works, run it for the rest of the parsed decisions** — proving it once before scaling it up, same approach as every phase so far.

■ **What "done" looks like for this phase:** query the database for the Abu Dhabi decision, and the telemetry snapshot for both cars comes back attached to it — no manual script required, no re-investigation needed.

Technical Detail: What We're Starting From

Today's `penalty_decisions` table (from Phase 2) has no field for year, event, or telemetry at all:

```
CREATE TABLE penalty_decisions (
  decision_id BIGSERIAL PRIMARY KEY,
  source_file TEXT NOT NULL,
  driver_text TEXT,
  own_car INT,
  other_car INT,
  competitor TEXT,
  session TEXT,
  incident_time TEXT,
  turn INT,
  fact TEXT,
  infringement TEXT,
  decision TEXT,
  penalty_points INT,
  reason TEXT
);
```

Phase 4 will likely add something like this — new columns for the race identity, plus one to hold the telemetry snapshot itself (Postgres's `JSONB` type can store structured data like this directly in a column, without needing a brand new table):

```
ALTER TABLE penalty_decisions
  ADD COLUMN race_year INT,
  ADD COLUMN race_event TEXT,
  ADD COLUMN telemetry_snapshot JSONB;
```

The reusable function from step 3 will be built directly from what already works in `verdict/telemetry/explore_session.py` — the same `get_session` / `get_telemetry` / biggest-speed-drop logic that already found the real Abu Dhabi incident moment, just wrapped into a function instead of a one-off script.

A Couple of Terms, Explained Simply

Join — combining two related pieces of data (here: a penalty decision and its matching telemetry) so they live together as one record instead of two separate things you have to connect by hand each time.

Snapshot — a small, focused slice of data covering just the moment that matters, instead of saving everything.

JSONB column — a way for Postgres to store structured, nested data (like our telemetry snapshot) directly inside a single table column.

That's the plan. Once one decision has its telemetry permanently joined and queryable, we're ready for Phase 5 — the real ML phase, comparing incidents against each other to find where similar cases got different verdicts.